



# OKOPIPI

## Network Protocol-Centric Blockchain L1

Decentralizing the Cloud

Jeff Venable, Sr.

Confidential

### Abstract

Okopipi is an Internet protocol first approach to scaling blockchain performance and interoperability. Leveraging virtual network switches as nodes, it implements intelligent, efficient routing of transactions and consensus at the network layer with instant finality. Combined with containerization hosted by the virtual switch, language agnostic protocols enable arbitrary smart contract virtual machine execution and integration on a common ledger eliminating bridging, typically needed for blockchain interoperability, or rollups in the case of L2 scaling chains on top of base L1 chains. Furthermore, the network can host containerized workloads in the decentralized physical infrastructure edge creating the next generation of cloud. Data severability from cryptographic proofs enables selective disclosure and privacy at the node replication layer supporting the sovereign use case, all in a network supporting transaction throughput beyond a million per second.



Confidential

|                                            |    |
|--------------------------------------------|----|
| Abstract .....                             | 1  |
| Introduction .....                         | 4  |
| The Problem .....                          | 5  |
| Philosophical approach.....                | 6  |
| Efficient network protocols.....           | 6  |
| Severable data from security proof .....   | 6  |
| Network non-interactivity .....            | 6  |
| Mathematical determinism .....             | 7  |
| Security based on network randomness ..... | 7  |
| Nodes as network routers .....             | 7  |
| Ledger transparency .....                  | 8  |
| Contract transparency .....                | 8  |
| Finality and conflict resolution.....      | 8  |
| Architecture .....                         | 9  |
| Ledger and message serialization .....     | 9  |
| Schema .....                               | 9  |
| Severable data .....                       | 10 |
| Network topology .....                     | 10 |
| Overview.....                              | 10 |
| Nodes.....                                 | 11 |
| Contract validators.....                   | 11 |
| Block orderers.....                        | 11 |
| State archivers .....                      | 12 |
| Virtual switches .....                     | 12 |
| Multicast Replication .....                | 13 |
| Transaction lifecycle .....                | 13 |
| Initiation .....                           | 13 |
| Validation .....                           | 14 |
| Ordering consensus.....                    | 15 |

|                                                     |    |
|-----------------------------------------------------|----|
| Algorithm summary .....                             | 15 |
| Algorithm detail.....                               | 17 |
| Reliable multicast agreement .....                  | 17 |
| Threshold voting.....                               | 17 |
| Random deterministic permutation .....              | 18 |
| Conflict detection .....                            | 20 |
| Post-ordering predicates.....                       | 20 |
| Privacy .....                                       | 21 |
| Interesting enhancements.....                       | 21 |
| Contract network access .....                       | 21 |
| Selective data disclosure .....                     | 21 |
| Geofencing.....                                     | 21 |
| Ledger transparent events.....                      | 21 |
| Decentralized cloud.....                            | 22 |
| Blockchain based container registry services .....  | 22 |
| Media streaming.....                                | 22 |
| Use Cases .....                                     | 22 |
| DeFI.....                                           | 22 |
| Central Limit Order Book (CLOB) .....               | 23 |
| Interoperability Without Bridging.....              | 24 |
| Decentralized Cloud – Kubernetes .....              | 24 |
| Decentralized Physical Infrastructure – DePIN ..... | 24 |
| GPU Monetization for AI .....                       | 24 |
| IoT .....                                           | 26 |
| Space .....                                         | 27 |
| Tokenomics.....                                     | 28 |
| References.....                                     | 29 |



## Introduction

Blockchain was first invented as a byzantine fault-tolerant distributed ledger creating cryptocurrency. It focused on secure cryptographic primitives powered by a network-based approach to decentralize governance and control through automated programs coined “smart” contracts. The first blockchains were successful at focusing on decentralized finance, but did not reach their full potential as they are constrained to the application layer of a network. The decentralized nature of low-level Internet protocols has never been integrated as a primitive building block to drive the technological stack above it. This oversight in ignoring what made the Internet tick creates both a performance and interoperability obstacle that fundamentally limits the scale of what blockchains can achieve.

What is envisioned is pushing the function of consensus down into the Internet protocol stack itself, using techniques that it and later the Web spent decades refining to enable efficient routing of information at planetary scale. Furthermore, in a global network of heterogeneous devices well-defined protocols are mandatory to enable interoperability. By defining all interactions with the blockchain Internet protocols and the ledger as a serialized data concern, language agnosticism is achieved making the choice of a virtual machine for smart contracts a severable security domain. Thus, any virtual machine can coexist on the same ledger, calling each other irrespective of smart contract programming language and ending the need for fragile bridging or roll-ups.

Finally, this innovation isn't limited to just a blockchain protocol. The network nodes themselves are virtual switches controlled by the blockchain, becoming the driving force behind an intelligent network design which routes consensus and information without depending upon wallets or clients. Services become containers hosted by these network switches, creating a software-defined approach that enables pushing a “cloud” to decentralized physical infrastructure, accessing global compute both large and small.

Confidential



## The Problem

Separate from Bitcoin and its underlying network of Proof of Work mining, the blockchain ecosystem has evolved on several fronts. Bitcoin created a decentralization phenomenon that has created more than 800 blockchains globally, each trying to innovate beyond Bitcoin and deliver a unique value proposition. This has given rise to several base L1 blockchains, and a continually growing list of L2 blockchains built on top of them. While the L1 blockchains deliver value and support for a growing number of use cases, L2 blockchains are designed to address challenges or restrictions of the underlying L1 blockchain. At the same time, these L2 blockchains have cultivated their own developer communities resulting in bespoke ecosystems.

After nearly 10 years of development by innovators in this space, many are concluding that the current stock of blockchain networks and protocols have reached a fundamental limit in terms of performance, scalability, security, and interoperability. The current blockchain protocols are too far removed from and have generally ignored or unappreciated the role and the function of the Internet. In short, many blockchain protocols have simply re-implemented what the Internet already does, but at the application layer. This is not only woefully inefficient, but leads to slower adoption, confusion among new developers and businesses looking to innovate, and barriers to unlocking the true potential of the technology. Not recognizing Internet fundamentals has further led to fragmentation of the ecosystem with chain specific programming languages with steep learning curves, abandoning non-web3 software developers.

The Okopipi Network unifies the development experience by being attractive to an estimated software developer ecosystem of more than 30M web2, cloud, and web3 software innovators.

Confidential



## Philosophical approach

### Efficient network protocols

**Problem:** Complicated communication mechanisms reliant on streaming to multiple network nodes.

**Solution:** Deconstructed messages focusing on how the network physically works, such as UDP datagrams instead of large streams, exchanged in a way that fault-tolerance is built-in and lossy networks are acceptable by design. Rely on switches to route traffic with intelligent replication to “multicast” at the network layer.

### Severable data from security proof

**Problem:** Backhauling vast amounts of data physically across a planet to achieve consensus, expecting every network node to have unlimited storage and resources. Expecting all nodes (regardless of ability) to store state, especially in emerging DePIN use cases.

**Solution:** The ledger is a Merkle Tree, originally designed to secure data integrity back in the 1970s when storage mediums were unreliable by detecting block-level corruption and enabling efficient verification and repair. This means that data can be left entirely off-ledger enabling privacy and may be selectively shown by participants either publicly or privately with the Merkle Tree authenticating its veracity. In a world where 99% of the players do not care about everybody else’s transactions, only that the ledger is correct, why have an expectation that they must store all the data instead of just its proof when challenged? Furthermore, the data itself is irrelevant to achieving security consensus on a separate Merkle Tree proof, so why backhaul it across the planet to a central decision-making point?

### Network non-interactivity

**Problem:** Having to communicate and synchronize between network participants creates critical sections of inefficiency and latency waiting for other parties to respond. Blockchain whitepapers typically reference “multicasting” data efficiently to multiple network node targets, but upon further review it is almost universally true to discover loops of “unicast” to each target sequentially. Further, the use of TCP streams, which is traditionally used to recover data lost in transmission, temporally stalls transaction submission repairing packets lost in the network further impeding performance.

**Solution:** Remove all client or node driven communication wherever possible. Messages are minimized to maximum transmission unit (MTU) datagrams where nodes in the Okopipi network efficiently “multicast” to all the required participants at the routing layer with built-

Confidential



in tolerance for message loss. Multicast is a traditional Internet communications mechanism where switches in a network propagate packets to multiple destinations without the involvement of the data source. The Okopipi Network is the first to truly use this mechanism across a wide area network (WAN) such as the Internet. Since packet loss is a feature with threshold quorum voting counted in aggregate signature collection, there are no stalls to repair lost packets when compared to traditional streaming protocols. When sufficient signature votes are collected, immediate progress without waiting for the rest is possible.

## Mathematical determinism

**Problem:** When quorums of participants must agree upon an outcome, they typically need to communicate.

**Solution:** As noted, communication creates inefficiency. Minimize communication to setting up a common starting point (a quorum commitment to a random **R** in an aggregating signature protocol) which can be asynchronously performed in advance of work and refactor the problem such that agreement is a deterministic outcome from there.

## Security based on network randomness

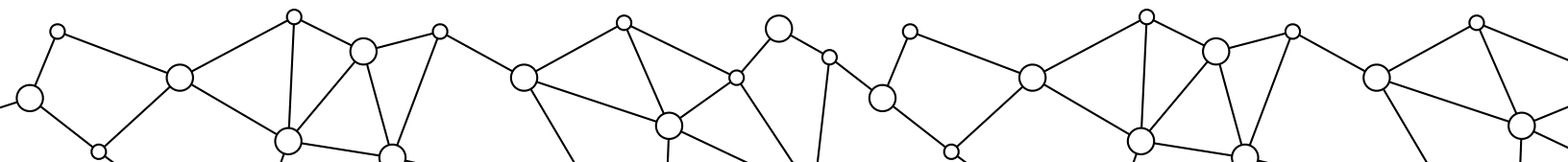
**Problem:** Cryptographic constructions and Byzantine Fault Tolerance are often overly complex, and reliable sources of entropy in a shared system are often lacking.

**Solution:** Data packets travel geophysical distances at the speed of light with indeterministic “hops” at physical infrastructure arriving at disparate destinations at unpredictable times. On a planetary scale no individual entity, or coordinated effort, can control this chaos. Okopipi is the first blockchain protocol to use the natural entropy of the Internet to protect itself.

## Nodes as network routers

**Problem:** Blockchain nodes today are services written at the application layer, oblivious to the network they are reliant upon.

**Solution:** Blockchain nodes become the virtual switches underpinning the network itself. Services become containerized workloads hosted by this software-defined network. Services are not limited to blockchain only concerns enabling, for example, the creation of a “cloud” in a decentralized physical infrastructure world.





## Ledger transparency

**Problem:** Figuring out what the present state of an account, balance, or other concern is elusive and difficult. What a transaction “did” is not readily clear, making questions around security and side-effects a genuine issue. Entire third-party companies and solutions are orchestrated around uncovering “what actually happened”.

**Solution:** Standard serialization protocols based on published schemas enable reflective inspection of ledger state. Any stateful object such as a transaction or one of its outputs is easily locatable and readable in human-friendly (such as JSON) or automatable formats.

## Contract transparency

**Problem:** There is no standard schema publication of what a contract is. Methods may not be explicitly declared (or even maliciously hidden). There is no “contract” to a contract that guarantees what its external behavior is.

**Solution:** Standard serialization protocols enable Remote Procedure Call (RPC) semantics through published schemas which enables any language to recognize and interoperate with the methods a contract supports calling. Schemas inherently are self-describing by supporting reflection, and virtual machines can prohibit behaviors not explicitly declared. Standards can be declaratively defined (e.g., ERC20) and enforced by virtual machine execution. As with traditional software defined network models, services are discoverable where each expresses their capabilities. This is not only immensely valuable for contract-to-contract interactions, but contract-to-cloud interaction for off-chain events, and contract-to-agent interactions for agentic AI use cases.

## Finality and conflict resolution

**Problem:** Blockchains tend to optimistically construct blocks using methods which enable the possibility of forking and heuristically determined finality. Conflict detection and similarly finality are typically left as an exercise for the client.

**Solution:** Conflict does not exist until the point in time at which a block order is chosen. It's irresponsible for such ordering to leave the rest of the network guessing at the finality of its decision. [Multi-Version Consistency Control \(MVCC\)](#) implemented at the ordering service, mapping global state to versions, enables conflict detection once a block order is chosen. MVCC is a common mechanism of version management for digital artifacts. Invalid transactions can be deterministically rejected to a second part of the signed block providing instant finality.





# Architecture

## Ledger and message serialization

### Schema

From [Cap'n Proto](#):

*Cap'n Proto is an insanely fast data interchange format and capability-based RPC system. Think JSON, except binary. Or think [Protocol Buffers](#), except faster.*

It was invented by [Kenton Varda](#) (and friends), one of the original authors involved in creating Protocol Buffers at Google. However, for blockchain it is superior because it supports canonical message format (suitable for cryptographic hashing and signing) and its capability RPC security model is interesting for supporting persistent and resumable token state.

The network will publish system schemas in the ledger as part of its genesis and thus will be self-describing. Tooling will exist to convert any binary message into human-readable form, such as JSON. This means that any block, transaction, event, or other object can be interactively inspected by either humans or automated systems as needed.

Here is a pseudo-example of the textual code representation of what such a schema might partially include:

```
struct Call {
  union {
    rpc                @0 :RPC;
    data               @1 :Data; # EVM compatibility, etc.
    struct RPC {
      contract         @0 :UInt64;
      method            @1 :UInt16;
      params            @2 :AnyStruct $kind("kind");
      kind              @3 :UInt64;
      receipts          @4 :List(Receipt); # selective disclosure
    }
  }
}

struct Transaction {
  from                @0 :SID; # account object
  to                  @1 :SID; # account object
  timestamp            @2 :RFC3339; # e.g., "1990-12-31T23:59:60Z"
  call                 @3 :Call;
  merkle               @4 :List(MerkleNode);
  signature            @5 :Signature; # merkle[0]
}
```

Schemas are compiled into a binary form serialized as a Cap'n Proto message as well. By requiring a contract to publish a schema and its RPC methods as part of its creation, the blockchain ledger effectively has a reflection registry by which any network participant can dynamically introspect the contract and figure out how to interact with it, and any virtual



machine can confirm only correct and published contract behavior, such as strongly typed inputs, outputs, events, etc.

Serialized messages have a standardized binary encoding, which means any programming language can interact with it. There is a plethora of [libraries and tools](#) supported from the open-source community for this exact purpose. In the pseudo-schema above the **Call** object supports a choice between **Call.RPC** and **Call.Data** method for invoking a contract providing an interoperable means of invoking an EVM contract using its traditional 'calldata' input parameter rather than a traditional RPC.

Message serialization and RPC are a standard which is widely used in cloud and enterprise software and offers maximum interoperability and developer ecosystem access.

### Severable data

One can see that 'objects' are described by such a schema, meaning that a cryptographic proof in the form of a Merkle Tree can be constructed of the same form which confirms each field of the object. Partial or fully redacted objects may be stored or transferred across a network in a manner where missing fields are substituted with their respective hash nodes from the validating Merkle Tree, thus proving the correctness of any data fields which are visible. This also enables reducing most network messaging to singular datagram (UDP) packets. Packet loss tolerance is built into all network constructs. This is a key differentiator, separating contract execution from block ordering as two separate phases avoids back-hauling state data across the planet as it has no influence on choosing random block ordering!

## Network topology

### Overview

The main design is to separate contract execution validation using an infinitely horizontal-scaling network. Client wallets send transactions to the nearest ingress node (virtual switch acting as a network router), finding it via [global server load-balancing \(GSLB\)](#) like [content delivery networks \(CDN\)](#) for the web. The network itself will multicast replicate the transaction hop-by-hop to two (or more) deterministically random consensus quorums which have a  $(2F+1)$  byzantine fault tolerant intersection of nodes, where (F) is the number of faulty nodes in any quorum. The wallet will collect aggregating signatures from the execution nodes to form a group signature validating the transaction.

Since ordering is separate from contract execution, the next challenge is to vertically scale ordering consensus (the main critical section of the network) to achieve maximum transaction throughput. The network multicast replicates all execution signed transactions



to a random quorum of ordering nodes. It is important to note that this replication avoids backhauling transaction state data across the planet by separating it from the endorsed merkle-proof, enabling efficient single datagram (UDP) packets only. The quorums change randomly in epochs, each of which consists of one or more blocks for some transient degree of network routing stability and to evade [distributed denial-of-service \(DDoS\)](#) attacks. The ordering nodes:

1. receive signed transactions from network multicast replication independently
2. confirm transaction signatures at packet line-rate admitting them to a memory pool
3. use “reliable multicast” to exchange transaction hash values
4. perform threshold voting on transactions collected
5. produce a block of randomly ordered transactions through deterministic mathematics that clients may later verify
6. mark failed transactions induced by ordering providing instant finality
7. collect aggregating quorum signatures at a fault-observing future epoch quorum (guaranteeing liveness) which announces the block

## Nodes

There are three kinds of nodes in the network:

1. Contract Validators
2. Block Orderers
3. State Archivers

The tokenomics of funding these nodes may differ by use case. Decoupling these network functions allows the overall construction to focus more tightly on efficient design and node requirement flexibility.

### *Contract validators*

Contract validators are not expected to be powerful, with an ideal target being single-board computers such as Raspberry Pis, allowing everyday enthusiasts to take part in the network; c.f. [validation](#). Contract execution is the most latent part of the framework since said execution time may be arbitrary (within constraints).

Contract validators expect to earn a part of the transaction costs that are successfully included into blocks.

### *Block orderers*

Block orderers are expected to require significant hardware and are ideally co-located at key Internet data exchanges (IXPs). This is due to the desire to achieve line-rate packet processing to achieve block transaction throughput above one million per second on



100gbps links. This part of the network is isolated in function to emphasize a focus on vertically scaling this throughput.

Block orderers expect to earn a part of the transaction costs contributing to the blocks produced.

### *State archivers*

State archivers focus on long term data storage and retrieval. State may be accessed via network protocols using flat ledger addressing; c.f. [Validation](#). Contract execution or other network nodes may request any publicly hosted information from archive nodes by sending a singular packet message with a state identifier or flat ledger address. Any nearby archive node may respond with the requested information.

These nodes may expect token compensation in the form of storage monetization when initially hosting data and in fees when retrieving it for contract execution.

### Virtual switches

Every node in the network is a virtual switch based upon open source; [Vectorized Packet Processor \(VPP\)](#). The network protocols needed are aimed at classic Internet style standardization such as [IETF RFCs](#).

Each node is expected to be able to isolate network data traffic away from the underlying host system. In fact, through use of a [Docker Network Driver](#), the virtual switch can create private networks on which it can host containerized services secured by the blockchain ledger. [An open source example](#) (that this author mentored) was used in blockchain enabled Internet of Things (IoT) environments.

Every node creates a public key as its identifier, serving a dual-purpose for the virtual switch to create IPsec tunnels as routes between nodes in the network to achieve a desired cardinality in a self-organizing mesh. This means inside the virtual network the entire infrastructure is encrypted end-to-end with hop-by-hop tunnels.

Nodes coming online advertise themselves as available to the network for a leased duration, like DHCP leases when devices attach to Wi-Fi networks. Leases are created via a system contract transaction where they advertise the node's key, network address, function, etc. We will see later how block epochs organize a time window of these leases of availability into quorums at each stage.

Finally, the virtual switches also allow non-IP based network transport such as deploying ordering nodes at data exchanges or archival nodes at CDN datacenters running for example the Border Gateway Protocol (BGP) on the backbone of the Internet itself.

## Multicast Replication

A new protocol, like [Bit Indexed Explicit Replication \(BIER\)](#), is used to achieve efficient multicast replication to quorums of nodes in the network. Using a routing header inserted into the network packet, each node's virtual switch can make routing decisions on which IPsec tunnel paths to copy packets to.

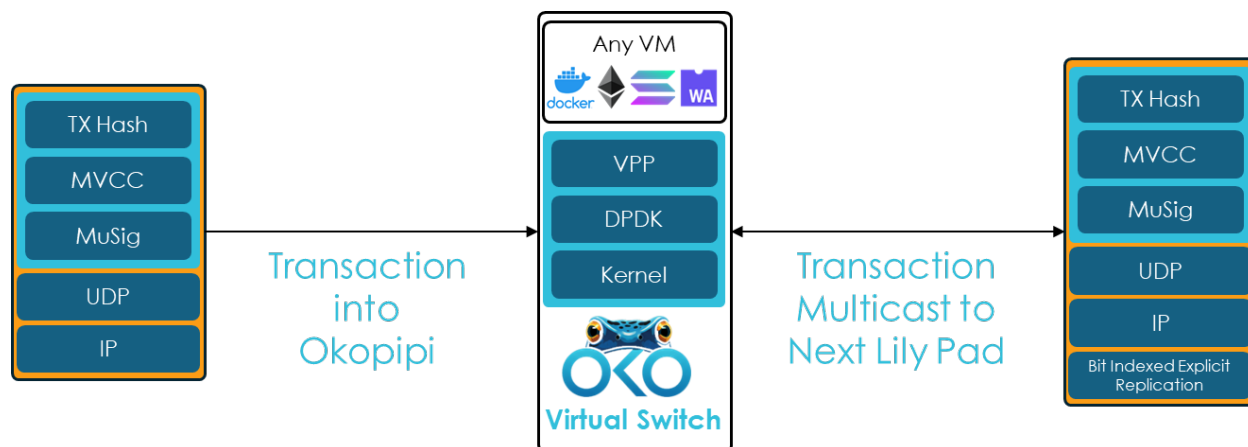


Figure 1: Okopipi virtual switch used to receive transactions from the wallet and intelligently replicated them across the validation node groups randomly selected in each epoch.

This means in practice if a wallet sends a validated transaction message to an edge, ingress gateway, that gateway and later hops in the network are capable of efficiently copying the message in a minimally branching topology tree to reach an arbitrary quorum of nodes in the network. Furthermore, communication within a quorum is efficiently achieved in a similar mechanism by sending individual packets to the underlying switch for replication.

## Transaction lifecycle

### Initiation

A wallet library constructs a transaction message by filling in the fields described by the transaction schema, building a Merkle Tree hash verification, and signing it. The full transaction message need not fit within a single network datagram MTU (maximum transmission unit) of ~1500 bytes, as data may be redacted at some later point to leave only the required proof fields reducing the message below the size of a single datagram. This enables a wallet client the flexibility to send large transactions over streaming (HTTP or TCP) communication channels to start transactions, while the network reduces later work to a simple efficient packet.

The wallet client finds its nearest ingress gateway for the network via DNS based [Global Server Load Balancing](#) and sends its transaction message to it. The wallet client does not

require configuration of specific network node identifiers, it only requires the name of the network like any common web browser does when verifying TLS certificates (e.g., HTTPS).

Upon receipt of a transaction message at a network ingress gateway, the transaction signature is verified, or the message is rejected. Two later stages follow: (1) contract execution validation and (2) ordering in a block.

## Validation

The blockchain is divided into a series of epochs which may be defined as one or more numbered blocks of history. Each epoch all network nodes have advertised their address and presence in the lease protocol in an earlier epoch. This windowed state pool of nodes is divided into byzantine masking quorums, with a tunable degree of Byzantine Fault Tolerance; e.g., the “M-Path System” [1]. These quorums may be general purpose for any contract, or they may be assigned to specific contracts via deterministic means to achieve archive data locality or private subnets for data privacy.

The root merkle hash of a proposed transaction is used as an oracle to select validating node quorums to execute the contract, which upon successful completion issue their own signature on the transaction. The wallet client collects these signatures in a voting mechanism until a BFT quorum is achieved, producing an aggregate signature such as a [Bitcoin MuSig](#). The network ingress router multicast replicates the transaction message across the network to the identified quorums.

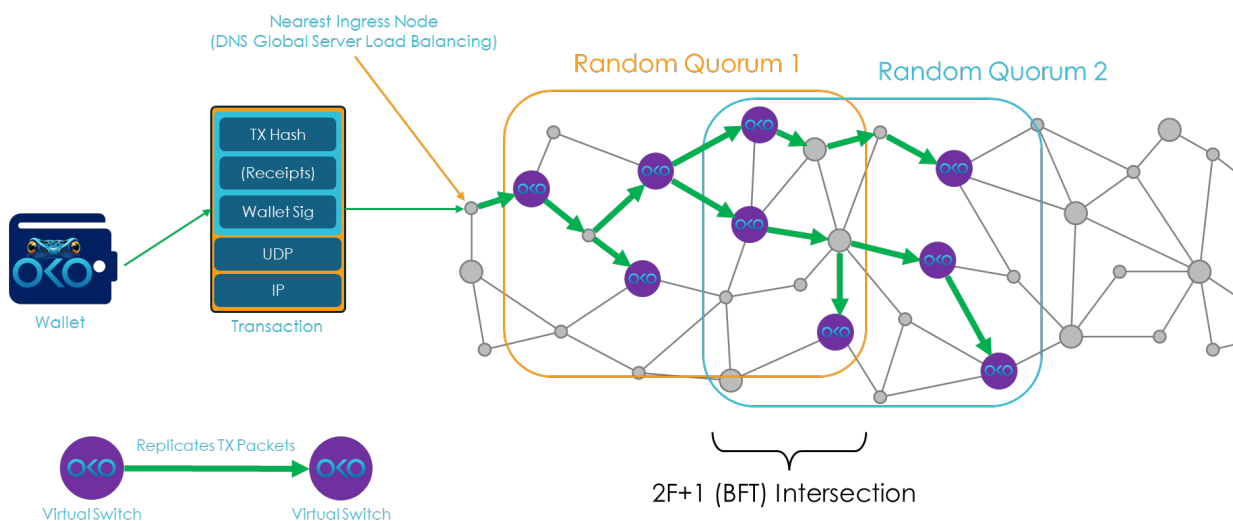


Figure 2: Random quorums via intelligent transaction packet replication with BFT intersection.

All hashes in the network are [Blake-3](#), a NIST finalist for SHA-3 and quantum resilient. It is faster than MD5 and implemented using merkle trees which allows for streaming verification and incremental updates.





It is possible for a quorum to recognize itself as a byzantine majority, but it would need to overcome the BFT tolerance parameter of the node intersection with secondary (or even tertiary, etc.) quorums. The pre-image resistance of the transaction hash plus a short-lived time window from the transaction timestamp (liveness) precludes gaming quorum choice from the client side.

The nodes which form the transaction validation quorums need not be powerful. It is expected that even inexpensive single board computers (such as Raspberry Pis) can provide a massively horizontally scaling node pool for the network. Furthermore, it is not required that the nodes keep ledger state in directly attached storage.

Any state object in the network has a unique state identifier (SID) which maps to a ledger Address for its “HEAD” version. The SID is name spaced by the contract GUID and a storage slot number (e.g., SID = {GUID, SLOT}). An address is “flat” and finds a ledger location in O(1) lookup where its current value may be found; c.f. [Distributed Ledger Addressing](#). Such a trivial address is suitable for efficient broadcast in a network to nearby archival nodes which respond to the validator with the requested information. If different validators in a quorum wind up with stale state version views from different archives, their voting signature won’t contribute to the threshold of the quorum. Furthermore, Cap’n Protocol block messages can be efficiently mapped into virtual memory by archive nodes, page faulting in when requested in O(1) access. We will see later that the block ordering mechanism will also detect and reject stale state version conflicts if the validating quorum has a stale view.

## Ordering consensus

### *Algorithm summary*

Once a wallet client collects sufficient voting signatures from contract validation quorums into a verifiable aggregate signature, it has two choices. It can withhold the proof of the transaction with a time lock to implement constructions such as lightning network escrows, or it can send it before its expiration to a network ingress gateway for replication to the current ordering quorums to include into a block.

Ordering quorums are chosen with a known network function from earlier block hashes during epoch advancement, like how contract validation quorums are chosen.

Ordering quorums receive replicated transaction messages from the network and are expected to use more powerful hardware with the [Intel Dataplane Development Kit \(DPDK\)](#) driving the virtual switch to achieve packet rates of [8+ million packets per second \(mpps\) per CPU core](#). Using [batch cryptographic signature verification](#), prototypes using 4-core Intel Broadwell NUCs verifying more than 110K transactions per second have been demonstrated. There are alternative implementations of batch verification in other



languages such as Rust, and CPUs have advanced since the first prototype. The demonstrated limit reached was a result of 1gbps Network Interface Card (NIC) saturation, not a CPU bottleneck. Assuming approximate verification rates of 65K/sec on a single CPU core, it's easy to imagine what throughput an AMD EPYC processor with 192 cores might achieve.

The block ordering consensus algorithm is uniquely designed with non-interactive and deterministic results that are network efficient. It proceeds as follows:

1. A group **R** random suitable for use in an aggregating threshold signature (e.g., [Bitcoin MuSig](#)) is generated; this is done at some arbitrary earlier time and asynchronously, so it is not considered 'interactive'.
2. Transaction MTU datagrams (proof and state versioning changeset only, not including state data) are replicated by the network across the planet to quorums of ordering nodes.
3. These transaction messages are verified for proper processing by valid contract validation quorums for correctness at network rates that may achieve 100Gbps and are admitted to a memory pool which sorts them by hash number.
4. The quorum nodes achieve agreement on which members have received which transaction hashes via reliable multicast (broadcast); atomic broadcast (with total order) is not a required agreement construct (thus unlocking scale) as hash values are idempotent and their order is not yet relevant.
5. The reliable multicast agreement is further replicated by the network to future ordering quorums as failure detectors and to guarantee liveness should the current quorum fail. This is a required confirmation step as the future quorum either collects the aggregate signature for the block and announces it for the current quorum or transitively assumes ownership for liveness with the agreement of its next quorum.
6. Like the [TCP Nagle Algorithm](#), once too many transactions exist in the memory pool, too much time has elapsed since the last block each quorum member counts 'votes' of transaction hashes each member has; hashes which achieve threshold are included in the block and the rest are left in the memory pool. Faulty or latent nodes (e.g., extreme distance) are removed by the fault observer process.
7. The **R** commitment of the group aggregate signature is used to choose a permutation of the transaction hashes into a deterministically provable random order for the block. The group signature under **R** and the permutation are a double commitment to the block which any network client can repeat and verify.
8. Transaction ordering is checked for induced conflicts using a lookup of each state variable reference in the read and write sets (SID = {GUID, SLOT, VERSION}) at the

Confidential





head of the blockchain state (i.e., current block – 1). Failed transactions are marked providing instant finality.

9. Signatures are collected by a  $E+n$  epoch quorum which is on standby and receiving copies of all multicast quorum traffic as fault observers to ensure network *liveness* should the current quorum fail, adding their signature and broadcasting the final block to the network.
10. Blocks network broadcast to extend the Merkle hash tree of the ledger, thus generating future epochs and quorum selection. The routing tables of all nodes update in real time after block verification, allowing rewriting and redirection of stale packets already in flight towards expired quorums making the network intelligent.

### Algorithm detail

#### Reliable multicast agreement

Using the efficiency of network replicated multicast, ordering quorum members batch verified transaction hashes as they arrived in their memory pool into simple UDP datagram packets of up to 1500 bytes and send them to every other member, plus added fault observer quorums in future epochs. Given that a Blake3 hash is 256 bits long (32 bytes), it is possible to include a simple header plus approximately up to 40 hashes in a single message. The batch of hashes is hashed to produce a signature key. Every other node is expected to respond with a multicast acknowledgment message that is multicast similarly. Each member of the current quorum and the fault observer quorums track this state machine.

Given the timestamps of these back-and-forth messages a dataset of average ping latency between members is established. This makes it possible for current quorum members and the fault observers to notice when a member has ceased responding or otherwise functioning properly. Such a detection will trigger an agreement message calling out the faulty node to be removed from the quorum.

The multicast network underlay which connects these quorums is separate from the larger blockchain network, created using an alternative subnet of IPsec tunnels and routes. Since the tunnels are authenticated once using the node public key to set up the session, all messages are then authenticated without having to use added cryptographic overhead from signing the packets.

#### Threshold voting

Since every node is broadcasting its verified hashes in its memory pool to every other node with authenticated channels and acknowledgements, reliable multicast is achieved, and each node has an eventually consistent state machine of every node's memory pool. Failure to acknowledge messages removes nodes from the quorum's state machine.



Block production is triggered when one of four conditions occurs:

1. Some limit on the number of transactions is reached.
2. Some limit in elapsed time is reached.
3. The quorum is reduced to the minimal fault tolerant threshold.
4. The quorum fails and the fault observer future quorum intercedes (liveness).

Condition (1) and (2) is like the [TCP Nagle Algorithm](#). Condition (3) strives to produce accomplished work before quorum failure. The block will be marked to show an early termination of the quorum and handoff to the next epoch to reestablish quorum resiliency against faults. Condition (4) is asserted like (3), except by the next quorum, in which case the signatures are produced by the quorum from epoch +1 and collected by epoch +2, etc.

The block production is triggered by any node recognizing one of the four conditions with a specific message that includes the highest signature key of hash messages for each member from its view. Every node multicast acknowledges this message with its own view of highest signature keys of hash messages for the other nodes. Essentially, over time most of the messages have been fully acknowledged in the respective state machine views. Only the most recent trailing messages may still be in flight and unacknowledged. This allows each member to non-interactively compute the common view from which to perform transaction hash vote tallying.

Transactions to be included in the block are those which exist in a threshold fault tolerant majority in the quorum, and all members can compute this non-interactively (without further communication).

#### Random deterministic permutation

To produce a random order for the transactions in the block, the verifiable group random value **R** produced for the block signature is reused as a double commitment to the order of the block. With the group random as input, the goal is to produce a verifiable random order deterministically such that all nodes may compute it independently without further interaction. Furthermore, clients will be able to repeat this process to verify random ordering.

Observe that transaction hash values are traditionally a string of binary octets, making them a positive integer where  $(h \leq 2^n$  with zero-padding if needed). Hash values are also [natural numbers including zero](#) ( $\mathbb{N}_0$ ), which is a subset of the integers ( $\mathbb{Z}$ ). For every positive natural number ( $n$ ),  $(\mathbb{Z} \bmod n)$  forms a [finite cycle group](#) under addition denoted  $(\mathbb{Z}/n\mathbb{Z})$ . We can thus use [Galois Fields](#) for hash values, e.g.,  $\text{GF}(2^{256})$ . Any positive natural number ( $i$ ) is a generator of this group if it is [relatively prime](#) to the order of the group, which is  $(2^n)$ . A generator produces every element of the group by adding itself ( $k$ ) times modulo the order, the most



trivial example being the number (1). Since any *odd* natural number is relatively prime to the group order, all odd natural numbers in the group are generators. It is interesting to note that the addition operation in  $GF(2^n)$  is the bitwise-XOR operator, and converting any random integer into an odd number is a simple bitwise-OR operation with (1), making it extremely efficient to implement!

**Theorem:** any odd integer in  $GF(2^n)$  is a generator under addition.

**Proof:** Let (a) be an odd integer in  $GF(2^n)$ . Since (a) is odd, it is relatively prime to  $(2^n)$ . Therefore, there exists an integer (k) such that  $(a*k \bmod 2^n = 1)$ . This means that we can generate (1) by adding (a) to itself (k) times. Since (1) is a generator, we can generate any element of  $GF(2^n)$  by adding (a) to itself repeatedly.

Let's look at a trivial example:

$$\mathbb{N}_0/8 = [0..2^3] = \{ 0, 1, 2, 3, 4, 5, 6, 7 \}$$

Picking a random starting point (4) and an *odd* relative prime (5) used for addition, we obtain the permutation:

$$\{ 4, 1, 6, 3, 0, 5, 2, 7 \}$$

This is true for any random starting point and *odd* number in  $GF(2^{256})$ .

This hash property is useful because it means using the cryptographic signature commitment **R**, every member of the quorum can deterministically and non-interactively:

1. Sort the sparse hash set.
2. Choose the same random starting point; there are  $2^{256}$  such choices!
3. Create the same random relative prime in  $GF(2^{256})$  by picking any integer and deterministically bitwise-OR the integer (1) to guarantee it becomes odd with minimal perturbation; there are  $2^{255}$  such choices!
4. Use a [skip-list](#) data structure to index the sorted sparse hash set which has the property of implementing an iterable 'next' element if the current chosen element is not a member of the sparse set.
5. Remove the chosen element (or its 'next' neighbor) from the sorted sparse set.
6. Add the chosen element to the block.

Every network client verifying the block can:

1. Verify the ECC group signature.
2. Sort the transaction hashes.
3. Use the signature **R** value to repeat the deterministic permutation to verify order.



Sorting hashes can be [accelerated by hardware](#) using AVX [bitonic sorting networks](#) in constant time.

### Conflict detection

Conflicts may arise post-ordering. The ordering quorums keep an efficient state database in memory by mapping every state variable GUID in the universe to its head version and ledger address. Earlier network dataplanes I have implemented using huge-page memory to minimize CPU [Translation Look-Aside Buffer \(TLB\)](#) thrashing with [Hopscotch hashing](#) tables on [Intel Haswell CPUs \(circa 2013\)](#) have exceeded lookup read throughputs of more than 20M/sec on a single CPU core; it can only be faster on modern hardware! Billions of table entries should easily fit into a limit set of GB/TB sized physical memory.

This in memory lookup table allows the ordering quorum to scan the resulting block order for any induced conflicts prior to signing, rejecting and moving such transactions to a latter half of the block encoding as firm denial giving us instant transaction finality!

### Post-ordering predicates

Liquidity exchanges try to achieve intra-block state dependency computation via slippage mechanisms which fundamentally serializes and limits transaction throughput and is limited to block granularity. In this network, transactions may additionally serialize predicate conditions which abort a transaction post-ordering on *primitive* (not redacted and small enough to include in the transaction MTU packet) state the orderers can afford to cache alongside the versioning lookup tables. For example, reject a transaction if a *float32* balance goes below zero because of its ordering after other transactions in the same block. This mechanism satisfies another requirement for achieving instant transaction finality and simplifies contract obligations in a way that can be natively integrated into their programming languages yet described in a network protocol. This mechanism also enables contractual requirements for withheld transactions with time-lock escrow semantics sent to ordering later; e.g., lightning networks.

It may be possible to incorporate zkSnarks or homomorphic encryption for simple primitive tests (e.g., value is greater than zero) without exposing state values for these predicate tests. Such work is left for future research.

Confidential



## Privacy

## Interesting enhancements

### Contract network access

Because the network implements its own virtual network, contracts may be executed in an encrypted and closed virtual network in addition to a bytecode-based virtual machine with limited access to other well-defined services. Examples include restraining data requests to archival nodes within a limited geophysical “hop” distance, implementing geofencing concerns, or providing oracle services such as IoT device sensors in a DePIN environment where contract validation quorums can agree on data observed. An obvious implementation candidate is a contract implemented in WASM with host API access to WebSockets.

### Selective data disclosure

Data is severable from Merkle Tree proofs by default. The use case can decide whether to publish publicly by using tokenized storage from a network archival node, or uploading to [IPFS](#), etc. Banking or medical institutions can rely upon a public ledger of proofs yet exchange transaction receipts handling sensitive data privately. Wallets can keep their transaction receipts privately in perpetuity; if they lose their receipts due to insufficient redundancy it's nobody else's concern to store it.

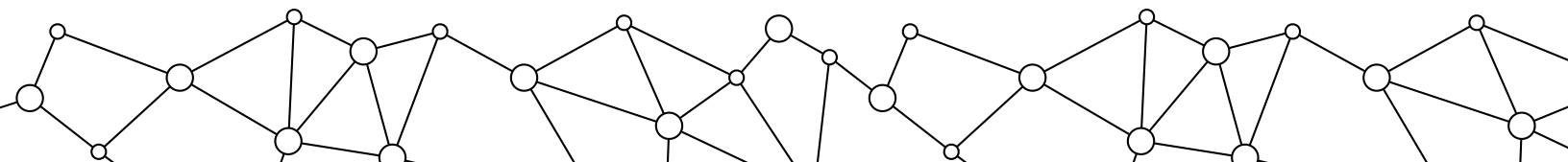
### Geofencing

Combine true network control with selective data disclosure. Nodes can enforce geofencing and [GDPR](#) like legal systems. Furthermore, this can be used to establish sovereign monetary systems like Central Bank Digital Currencies, offering complete control over data and transaction replication.

### Ledger transparent events

Since contracts are self-describing in their published serialization schemas and their state objects support reflection, it's trivial to implement subscription webhooks monitoring for state changes with lambda or conditional predicate support that trigger when new blocks arrive, assuming state is visible at archival nodes and isn't redacted.

Confidential





## Decentralized cloud

Network nodes are virtual switches with docker network drivers. Hosting arbitrary pod workloads via controllers based on blockchain secured configuration (e.g., contract state schema objects converted to YAML) in private encrypted network namespaces is a direct outcome. Nodes can mine not only network functions such as contract validation, block ordering, or state archival, but they can also auction off their spare resources creating DePIN workloads such as GPU/NPU based AI training. Imagine a billion single-board computers like Raspberry Pis with integrated NPUs unfetteredly deployed outside of datacenters. Taking this a step further, and given the ability for the network to support the deployment of native containerized cloud workloads classically managed through Kubernetes, Okopipi supports the migration of existing machine learning and artificial intelligence workflows from the cloud to the edge without forcing developers to abandon the time, energy, and resources spent to build in the cloud.

## Blockchain based container registry services

Like the Docker Network Driver API, think Docker Hub or another artifact repository, on blockchain. Contracts may be declared to require an officially supported container image such as a WASM VM, or distributed workloads may deploy only authenticated images in their DePIN environments, geofenced to archival nodes at their location. Network nodes obeying system contract state configuration pull images from these services to deploy on their virtual switch.

## Media streaming

Consider nodes with added network interface cards (NICs), CPU cores and memory separate from core network function but still using the virtual switch and network based efficient multicast support of the existing network to deliver real time streaming worldwide. Imagine integrating Pay-per-View and online gambling contracts with real-time AI analytics using data from IoT devices worn by sports players constrained to the competition environment but authenticated for dissemination to viewers.

## Use Cases

### DeFi

Decentralized Finance use cases number in the thousands in web3, each with their own communities, liquidity pools, markets, and excitement. Building on the Okopipi network, DeFi applications and services would benefit immensely from the protocol's internet "first"





principles. From a user experience perspective, the network eliminates waiting times for confirmations, enabling near-instant interactions such as swaps, lending, and borrowing. Friction is further reduced resulting from bridgeless interoperability allowing users to interact with multiple blockchain ecosystems without relying on bridges or third-party intermediaries, that are often slow, costly, and vulnerable to exploits.

Security is further increased as a simple result of the removal of bridges where we have seen countless exploits resulting in significant losses for retail investors. Further, Okopipi's instant transaction finality ensures transactions are irreversible once processed, reducing the risk of double-spend or chain re-org attacks.

One of the more exciting benefits of building on the Okopipi network is the seamless integration of liquidity across multiple blockchain ecosystems resulting from the bridgeless design. This will result in increased capital efficiency of DeFi protocols while at the same time allowing users to access yield opportunities that are improved across chains without the complexity of bridging or splitting of assets. This makes DeFi more accessible to a broader audience as fees for bridges are effectively eliminated resulting in traders capitalizing more efficiently using the network's unparalleled performance and limitless scale.

## Central Limit Order Book (CLOB)

With the global emergence of many tokenized asset trading platforms, questions on performance and legitimate scale have generally gone unanswered by digital asset trading firms. Given many, if not all, of these are built using a classical or legacy blockchain design, answering the question on the upper limit of performance and scale has been difficult to date. Given Okopipi's Internet based principles, where performance increases with the scale of the network, market participants have access to actual real-time trading capabilities. Given the network low latency, high transaction volume support, orders are matched and executed in real-time. Instant finality of transactions flowing through the Okopipi network reduces uncertainty for traders, giving them confidence to execute even more trades.

Cross-market arbitrage becomes attractive on Okopipi as the networks performance and reduced overall costs resulting from bridgeless transactions creates opportunities across decentralized and centralized exchanges. This improves overall liquidity across the sector and helps address price stability of assets. Furthermore, the network offers the ability to unify liquidity pools across multiple chains which greatly enhances the efficiency of global order book use cases. All this combined creates a fairer trading environment that reduces,





if not eliminates, transaction front-running attacks, providing a more level playing field for traders.

## Interoperability Without Bridging

Interoperability has been a major challenge in web3. We have seen the emergence of application layer networks like Cosmos attempt to address interoperability as well as the emergence of many bridge Dapps. While Cosmos' efforts to address interoperability are commendable, the network still suffers from the same scale, performance, and security challenges that are affecting all other blockchains from achieving the kind of operational excellence that will be needed.

## Decentralized Cloud – Kubernetes

Containerized workloads using DOM object structures with controllers on nodes providing compute managing pod workloads on top of the virtual switch. Creating private namespace networks with access control routing.

Working with the world's largest Kubernetes community, Okopipi will be used as the defacto network enabling workload migration from the cloud to the edge. Hyperscalers like GCP, AWS, and Azure compete fiercely for workload migration amongst themselves with strong incentives in place for sales, technical sales, and partner staff. What is not part of their cloud economics is migration to a destination other than another cloud, colocation, or on-premises operating environment.

Companies have poured billions of dollars in going from being cloud ready, first workload moved from on-prem to cloud, to eventually running significant portions of their business in the cloud. Yet, no cloud provider will guarantee more than 3-9's availability and vendor lock-in is a constant risk. Okopipi's unique network attributes not only allow complete freedom to choose from tools and services built by millions of cloud developers, but also availability and uptime more than what cloud vendors offer today. Furthermore, as transactions between workloads and execution on the network are performed at the packet layer, interactivity with web3 projects becomes as simple as executing a peer-to-peer transaction, enabling a near infinite number of potential web2+web3 hybrid use cases.

## Decentralized Physical Infrastructure – DePIN

### GPU Monetization for AI

GPU marketplaces are a growing trend in web3, and one of the dominant use cases in DePIN. As the number of Bitcoins left to be mined becomes smaller and smaller and rewards for mining also shrink, miners are seeking to maximize returns on capital investments by

Confidential



shifting from Bitcoin mining to making hardware available for Decentralized AI use cases. Bitcoin mining chipsets, be they FPGAs, GPUs, or ASICs, are all practical for machine learning training, inference, and edge applications. A small industry is starting to appear where Bitcoin miners are opening their data centers to non-Bitcoin use cases. A survey of this emerging segment reveals that most projects try to shoehorn decentralized apps that speak the tokenomics of the underlying GPU market provider. What is missing is the massive community of non-Dapp projects that have yet to discover web3 that are deployed at scale across the cloud. Okopipi uniquely provides access to this untapped community where GPU providers can now make their resources available to a developer community 10x larger than that of web3.

## Decentralized AI

One of major gaps in AI solutions, both web2 and web3, is the validation and independent attestation of workflow efficacy and ethics. As mentioned earlier, there are an order of magnitude more web2 AI developers than there are web3 AI developers. Given the network naturally supports hosting cloud AI workloads with little to no friction, nearly a decade's worth of AI investments can efficiently be retargeted to Okopipi with ease. That said, efficacy and provable ethical sourcing of training data are still elusive.

Opportunities exist for all participants in the AI ecosystem when using the power of the Okopipi network. From source data, through inference at the edge, Okopipi can power the workflow end-to-end.

Data providers, those who manage the gold in all AI workflows, can leverage Okopipi's built in data confidentiality framework to monetize their painstakingly curated datasets with safety. Using the networks selective data disclosure mechanism, access to slices of data may be individually made available through tokenomics or in their entirety. The decision and control are completely left to data providers. This enables the creation of entire training data marketplaces, all on-chain with confidentiality, where archive node operators may provide specialized storage assets in compliance with the code of conduct established by data providers, encoded by smart contracts, or where the data providers may operate their own bespoke archive nodes as part of the okopipi network. Further, incentives can be created to make available metadata that can be remotely attested to interrogate the efficacy training data by anyone attached to the network

As models are created through both web2 and web3 means, the network enables entire model marketplaces to be established where consumption, deployment, and refresh are incentivized by tokenomics. This allows entire new consumption patterns to be created with model may be connected by smart contract pushing data in parallel or serialized for an



infinite number of combinations powering an entirely new class of network based inference. As we say in classical blockchain, the network is the database, in this case, the Okopipi network supports the development of the worlds largest inference engine. Models across multiple chains can be orchestrated using Okopipi's RPC and packet based designed enabling, true model heterogeneity, where efficacy can be continually interrogated; imagine AI models with D.ID

The network also allows the addition of low cost edge inference hardware to be attached where access for model deployment can be tokenized. Leveraging Okopipi's virtual switch paradigm, the edge can be powered by billions of low cost community powered devices for data aggregation, model deployment, inference, and insights extraction for use cases in medicine, energy, environment and sustainability, finance, and so much more.

The combination of low cost inference hardware, as enabled by Qualcomm, Intel and others, decentralized inference, and the power and scale of the Okopipi network, creates a foundation for an Agentic AI future where the role of the network is critically important. It is often misunderstood that the internet "sees" all traffic and all patterns first. Acknowledging this, the Okopipi network has direct visibility to all transaction packet flows, knows the location of all resources, and as a result offers a next generation internet experience where a truly dynamic and "software defined" global experience is unlocked when blockchain attributes are realized through the internet itself.

## IoT

The number of IoT use case possibilities is unbounded. From mobility, construction, smart communities, energy, and beyond, the application of Okopipi to IoT use cases enables higher fidelity, and more real-time applications to be build when compared to other networks.

Unique to Okopipi, vendors like Bosch, Scheider Electric, Cisco, Honeywell, SAP, Samsara, Oracle, Johnson Controls, and countless others can not only deploy their web2 containerized software assets, but also migrate cloud workloads ranging from data aggregators and pre-processors, machine learning based optimization models, to inferencing engines, and so much more. This becomes compelling with web3 developers as one of the missing key ingredients is access to high quality, accurate, timely, and continuous off-chain data that can be proven to be untampered and complete. Vice versa, web2 companies can now participate in web3 tokenimics without needing to understand what blockchain or smart contracts are, instead discover and transact as if they were simply adding yet another network type to their already expansive list of supported L0 network types.



Of particular interest are the carbon offset, renewable energy, and green trading initiatives that have sprung up globally. Many project have been developed in this space focused market making platforms and market places built to trade tokenized renewable and/or carbon assets. Project like those from Zero13, Guild1, and others have proven successful in on boarding and tokenizing renewables made available for purchase by consumers. Historically, this has generally been limited to fortune 1000 companies of buyers and sellers and not the broader global community of producers and consumers. Using Okopipi, virtually anyone will be able to be a producer/consumer of tokenized renewables, where the networks ultra-high performance and instant transaction finality offers the ecosystem a significant advantage when compared to other networks.

## Space

Non-terrestrial applications of blockchain technology have been explored since 2016, with numerous crypto mining satellites being launched into orbit over the years. While these projects have served marketing goals well, they have not evolved beyond this. Using Okopipi, developers of space applications can now leverage the network for enhancing the cybersecurity, data integrity, and command & control resilience of satellite to satellite and satellite to ground station communications.

With space assets now under attack by nation state adversaries and forming a critical component to global supply chain networks, powering proof-of-location and communication networks, agencies around the world are seeking solutions to protect and defend their satellite networks and the services they offer. Projects like Space Armour are building satellite platform and applications to provide more resilient command & control frameworks using blockchain technology. Leveraging the attributes of the Okopipi network, solutions like this would offer the most packet, energy, and photon efficient (laser based satellite to satellite communication is the most secure), data and command & control integrity capability.



## Tokenomics

Tokenomics for the Okopipi network are still in development. It is certain however that the network will be powered by a utility token to incentivize the operation and maintenance of the 3 node types, dapps, and developer incentives. A foundation will be created to manage grants and community engagements.

Confidential



## References

- [1] D. Malkhi, M. Reiter, and A. Wool, *The Load and Availability of Byzantine Quorum Systems*, SIAM J. Comput., 26 (2000), pp. 1889–1906.

Confidential